UNIVERSIDAD ORT
Uruguay

Facultad de Ingeniería
Bernard Wand-Polak
Cuareim 1451
11.100 Montevideo, Uruguay
Tel 902 15 05 Fax 908 13 70
www.ort.edu.uy

EVALUACION	**SOLUCION PARCIAL 1**	**GRUPO**	N4A-N4C-N4D	**FECHA**	18/09/2025
MATERIA	DISÑO DE APLICACIONES / DISEÑO Y DESARROLLO DE APLICACIONES				
CARRERA	ANALISTA EN TECNOLOGIAS DE INFORMACIÓN / ANALISTA PROGRAMADOR				
CONDICIONES	**- Puntos: 20** **- Duración: 2 HORAS** **- Sin Material**				

Ejercicio 1
Red de bicicletas compartidas

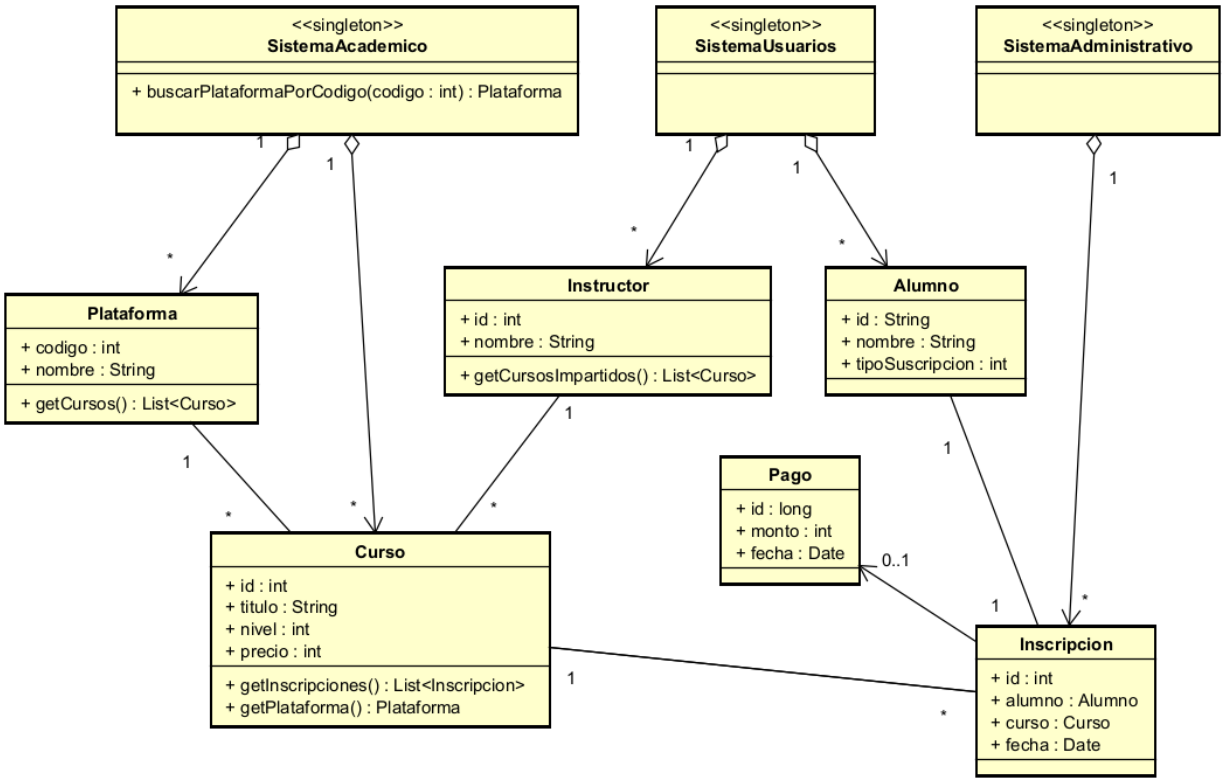
Una empresa opera una red de estaciones de bicicletas en varias ciudades. Cada estación tiene múltiples docks (anclajes) numerados y una capacidad máxima. Las bicicletas se identifican por un código único y pueden estar en una estación (ancladas) o en uso (prestadas por un usuario).
Los usuarios están registrados en el sistema. Cada usuario es dueño de un conjunto de bicicletas que comparte para que otros usuarios realicen viajes. Cuando un usuario realiza un viaje lo hace desde una estación origen a una estación de destino en una bicicleta especifica. Para garantizar la disponibilidad del servicio y la seguridad de sus usuarios se realizan mantenimientos a cada una de las bicicletas que forman parte de la red

Se pide:

- Elaborar un diagrama conceptual de dominio, incluyendo entidades, sus relaciones y multiplicidad.

Ejercicio 2
Sistema de gestión de cursos on-line

A continuación, se describe el modelo de clases actualmente disponible en el sistema (se detallan clases, atributos y los métodos públicos ya implementados —no es necesario implementarlos—. Suponga que todos los getters, setters y constructores están presentes.



Nota: El tipo de suscripción de un alumno es uno de los siguientes números enteros:

1 – Gratis: Se puede inscribir pero no genera pago
2 – Normal: Al inscribirse genera un pago con monto igual al precio del curso
3- Premiun: Al inscribirse genera un pago con un 15% de descuento respecto al precio del curso.

Parte a) Implementar los siguientes requerimientos a nivel del dominio:

Req.1: Dada una cantidad de alumnos N, obtener la lista de las Plataformas en las que cada uno de los cursos dictados tiene al menos N alumnos inscriptos.

```
public List<Plataforma> plataformasDondeTodosLosCursosTienenAlMenosNMatriculados(int n)
```

Req.2: Dado el código de una plataforma y un monto, obtener la lista de Instructores que, en esa plataforma, hayan dictado cursos cuyos pagos en total superen el monto indicado (suma de montos pagados en inscripciones de los cursos del instructor dentro de esa plataforma).

```
public List<Instructor> instructoresConIngresosMayoresQue(int codigoPlataforma, int monto)
```

Facultad de Ingeniería
Bernard Wand-Polak
Cuareim 1451
11.100 Montevideo, Uruguay
Tel 902 15 05 Fax 908 13 70
www.ort.edu.uy

Parte b) Dado los siguientes endpoints definidos en el controlador:

```
@RestController
@RequestMapping("/parcial")
public class Controlador {

    @PostMapping("/req1")
    public List<Plataforma> req1 (@RequestParam int cantidadUsuarios) {
        // implementar requerimiento aquí
    }
    @PostMapping("/req2")
    public List<Instructor> req2 (@RequestParam int codigoPlataforma, @RequestParam int monto) {
        // implementar requerimiento aquí
    }
}
```

Escribir lo necesario para que se pueden implementar los requerimientos definidos en la parte a.

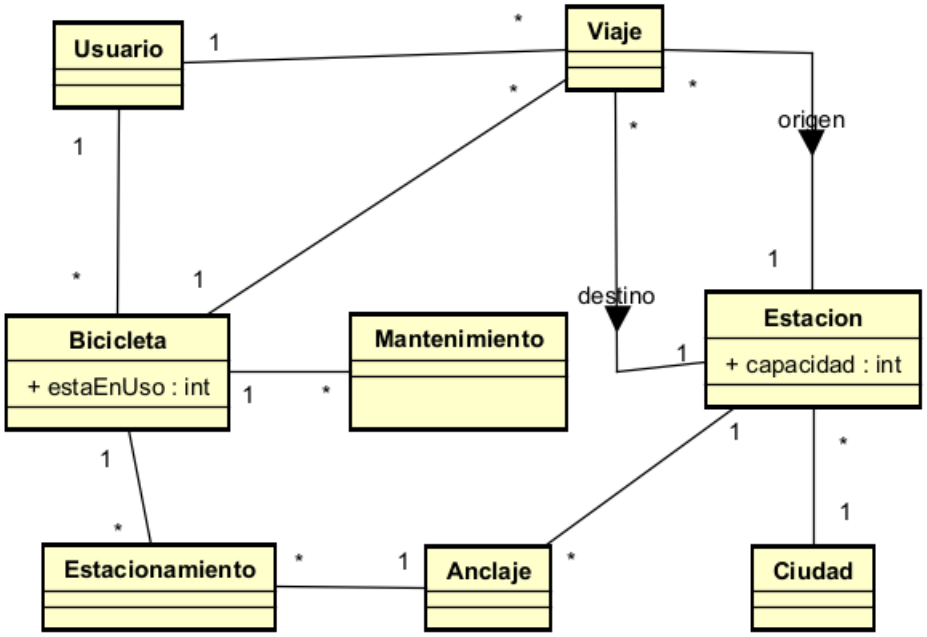
Consideraciones:

- Tener en cuenta que **todos los métodos que aparecen en el diagrama de clases ya se encuentran implementados** y algunos podrían ser útiles para su solución.
- **Utilizar estrictamente el diagrama de clases dado en la letra como base para su solución. Se podrán agregar nuevas clases, métodos y relaciones solo si son necesarios.**
- No es necesario implementar getters/setters.
- No puede haber lógica/código duplicado.
- **Debe aplicar el principio Experto**

Evaluación:

- Ejercicio 1: 8 puntos
- Ejercicio 2: 12 puntos

SOLUCION EJERCICIO 1



SOLUCION EJERCICIO 2 Parte a Requerimiento 1

```
// En SistemaAcademico : Retorna la lista de Plataformas
// en las que cada uno de los cursos dictados tiene al menos N alumnos inscriptos.
public List<Plataforma> plataformasDondeTodosLosCursosTienenAlMenosNInscripciones(int n) {
    List<Plataforma> resultado = new ArrayList<Plataforma>();
    if (plataformas == null || plataformas.isEmpty()) return resultado;
    for (Plataforma p : plataformas) {
        if (p.todosLosCursosTienenAlMenosNInscriptos(n)) {
            resultado.add(p);
        }
        return resultado;
    }
}
// En Plataforma: Retorna true si cada curso de la plataforma tiene al menos n inscriptos.
public boolean todosLosCursosTienenAlMenosNInscriptos(int n) {
   if (cursos == null) return false; // si no hay cursos, no cumple
   if (n <= 0) return true; // n <= 0 se considera cumplido
   for (Curso c : cursos) {
       if (!c.tieneAlMenosNInscriptos(n)) return false;
   }
   return true;
}
```

```
//En Curso : Determina si este curso tiene al menos n inscriptos.
public boolean tieneAlMenosNInscriptos(int n) {
   if (n <= 0) return true; // por definición, 0 o menos siempre satisface
   int contador = 0;
   return inscipciones.size()>=n
}
```

SOLUCION EJERCICIO 2 Parte a Requerimiento 2

```
// En SistemaUsuarios
public List<Instructor> instructoresConIngresosMayoresQue(int codigoPlataforma, int monto) {
        List<Instructor> resultado = new ArrayList<Instructor>();
        if (monto < 0) return resultado;
       Plataforma plataforma =
               SistemaAcademico.getInstancia(),buscarPlataformaPorCodigo(codigoPlataforma);
      if (plataforma == null) return resultado;
      for (Instructor i : instructores) {
         int ingresos = i.ingresosEnPlataforma(plataforma); // delegado al experto Instructor
         if (ingresos > monto) resultado.add(i);
      }
      return resultado;
   }
// En Instructor
public int ingresosEnPlataforma(Plataforma plataforma) {
   if (plataforma == null) return 0;
   if (cursosImpartidos == null || cursosImpartidos.isEmpty()) return 0;
   int suma = 0;
   for (Curso c : cursosImpartidos) {
      Plataforma pl = c.getPlataforma();
      if (c.getPlataforma() == plataforma) suma+=c.obtenerMontoTotalInsceipciones();
   }
   return suma;
}
//En Curso
public int obtenerMontoTotalInscripciones() {
          List<Inscripcion> inscripciones = c.getInscripciones();
          int suma = 0;
         for (Inscripcion ins : inscripciones) {
               suma += ins.getMontoPagado();
         }
        return suma;
}
```

Facultad de Ingeniería
Bernard Wand-Polak
Cuareim 1451
11.100 Montevideo, Uruguay
Tel 902 15 05 Fax 908 13 70
www.ort.edu.uy

```
//En Inscripción
public int ins.getMontoPagado(){
        int montoPagado=0
        if (pago!=null) montoPagado=this.getPago().getMonto();
        return montoPagado;

}
```

SOLUCIOON EJERCICIO 2 Parte b

```
@RequestMapping("/parcial")
public class Controlador {
   @PostMapping("/req1")
   public List<Plataforma> req1 (@RequestParam int cantidadUsuarios) {
            SistemaAcademico sa = SistemaAcademico.getInstancia();
           return  sa.plataformasDondeTodosLosCursosTienenAlMenosNInscripciones(cantidadUsuarios);
   }
  @PostMapping("/req2")
   public List<Instructor> req2 (@RequestParam int codigoPlataforma, @RequestParam int monto) {
      SistemaUsuarios su = SistemaUsuarios.getInstancia();
      return su.instructoresConIngresosMayoresQue(codigoPlataforma, monto);
   }
}
```